This 4-phase engineering, testing, and global compliance execution covers the **VibePitch Studio** repository architecture.

Phase 1: Codebase Audit, Bug Fixing & Refactoring

1. Static & Runtime Audit Summary

- **Inspected Modules**: Audited MainActivity.kt, VibePitchViewModel.kt, VibePitchRepository.kt, LocalGuestSessionManager.kt, GoogleAuthHelper.kt, VibePitchModels.kt, and all Jetpack Compose screens for unused imports, compiler warnings, null-safety edge cases, input bounds, and coroutine cancellation handling.
- **Schema & Bounds Hardening**: Enforced character length and numeric bounds in VibePitchViewModel and LocalGuestSessionManager so every prospect, lead, playbook task, and business profile created locally in Guest Mode passes firestore.rules schema validation during Day 20 or Day 30 cloud migration.
- **Legacy Test Cleanup**: Replaced template placeholder tests in ExampleUnitTest.kt and ExampleRobolectricTest.kt with full domain, session lifecycle, and ViewModel test suites.

2. Input Sanitization & Schema Validation Utility (InputBoundsValidator.kt)

Create or update the input sanitization module used by both VibePitchViewModel.kt and LocalGuestSessionManager.kt to enforce the exact firestore.rules constraints (businessName <= 100, contactName <= 100, websiteGapNote <= 300, tagline <= 150, notes <= 500, depositPercent in 0..100, setupFeeOffer in 0..50000, monthlyRetainerOffer in 0..5000):

```
package com.vibepitch.studio.domain

object InputBoundsValidator {
    const val MAX_BUSINESS_NAME_LEN = 100
    const val MAX_CONTACT_NAME_LEN = 100
    const val MAX_WEBSITE_GAP_NOTE_LEN = 300
    const val MAX_TAGLINE_LEN = 150
    const val MAX_NOTES_LEN = 500

    const val MIN_DEPOSIT_PERCENT = 0
    const val MAX_DEPOSIT_PERCENT = 100

    const val MIN_SETUP_FEE = 0
    const val MAX_SETUP_FEE = 50_000

    const val MIN_MONTHLY_RETAINER = 0
    const val MAX_MONTHLY_RETAINER = 5_000

    fun sanitizeBusinessName(input: String): String =
        input.trim().take(MAX_BUSINESS_NAME_LEN)

    fun sanitizeContactName(input: String): String =
```

```
        input.trim().take(MAX_CONTACT_NAME_LEN)

    fun sanitizeWebsiteGapNote(input: String): String =
        input.trim().take(MAX_WEBSITE_GAP_NOTE_LEN)

    fun sanitizeTagline(input: String): String =
        input.trim().take(MAX_TAGLINE_LEN)

    fun sanitizeNotes(input: String): String =
        input.trim().take(MAX_NOTES_LEN)

    fun clampDepositPercent(value: Int): Int =
        value.coerceIn(MIN_DEPOSIT_PERCENT, MAX_DEPOSIT_PERCENT)

    fun clampSetupFeeOffer(value: Int): Int =
        value.coerceIn(MIN_SETUP_FEE, MAX_SETUP_FEE)

    fun clampMonthlyRetainerOffer(value: Int): Int =
        value.coerceIn(MIN_MONTHLY_RETAINER, MAX_MONTHLY_RETAINER)
}
```

3. Coroutine Cancellation & Bounds Integration in VibePitchViewModel.kt & LocalGuestSessionManager.kt

Ensure coroutine blocks rethrow CancellationException cleanly and apply InputBoundsValidator before persisting to local guest storage or Firestore:

```
// Inside LocalGuestSessionManager.kt & VibePitchViewModel.kt
fun normalizeProspectInput(
    businessName: String,
    contactName: String,
    websiteGapNote: String,
    notes: String,
    depositPercent: Int,
    setupFeeOffer: Int,
    monthlyRetainerOffer: Int
): NormalizedProspectPayload {
    return NormalizedProspectPayload(
        businessName =
InputBoundsValidator.sanitizeBusinessName(businessName).ifEmpty {
"Untitled Business" },
        contactName =
InputBoundsValidator.sanitizeContactName(contactName),
        websiteGapNote =
InputBoundsValidator.sanitizeWebsiteGapNote(websiteGapNote),
        notes = InputBoundsValidator.sanitizeNotes(notes),
        depositPercent =
```

```
InputBoundsValidator.clampDepositPercent(depositPercent),
        setupFeeOffer =
InputBoundsValidator.clampSetupFeeOffer(setupFeeOffer),
        monthlyRetainerOffer =
InputBoundsValidator.clampMonthlyRetainerOffer(monthlyRetainerOffer)
    )
}

fun normalizeBusinessProfileTagline(tagline: String): String =
    InputBoundsValidator.sanitizeTagline(tagline)
```

Phase 2: Dependency & Architecture Optimization

1. Gradle & Version Catalog Audit (app/build.gradle.kts & gradle/libs.versions.toml)

- **SDK & Compatibility Matrix**: Preserved the verified Android SDK configuration (compileSdk 36, targetSdk 36, minSdk 24) alongside the Kotlin, Jetpack Compose, and Firebase BOM version catalog.
- **Dependency Pruning**: Commented out unused third-party libraries in app/build.gradle.kts to reduce APK footprint and accelerate Gradle build times.
- **Deprecation Sweep**: Confirmed zero deprecated Kotlin or Jetpack Compose API calls across all UI screens and helper classes.

```
// app/build.gradle.kts (Verified SDK & Pruned Dependencies Section)
android {
    namespace = "com.vibepitch.studio"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.vibepitch.studio"
        minSdk = 24
        targetSdk = 36
        versionCode = 2
        versionName = "1.1.0"
        testInstrumentationRunner =
"androidx.test.runner.AndroidJUnitRunner"
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
    kotlinOptions {
        jvmTarget = "17"
    }
    buildFeatures {
```

```
        compose = true
    }
    testOptions {
        unitTests.isIncludeAndroidResources = true
    }
}
```

Phase 3: Comprehensive Function Testing & Validation

1. Unit & Domain Model Test Suite (ExampleUnitTest.kt / VibePitchUnitTest.kt)

Tests all 7 NicheKey presets (HOME_TRADES, SALON_AND_CLEANERS, DENTAL_AND_TUTORS, AUTO_DETAILING, BOUTIQUE_FITNESS, ECOMMERCE_SHOPIFY, SAAS_AI_REMEDIATION), RevenueStreamMode mappings, PipelineStage transitions, NicheCatalog packages/add-ons, quote and deposit calculation math, and all 6 pitch/proposal/scope/referral script generators across OfferArchetype and ScopeGuardrailConfig:

```
package com.vibepitch.studio

import com.vibepitch.studio.domain.InputBoundsValidator
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertTrue
import org.junit.Test

class ExampleUnitTest {

    @Test
    fun
allSevenNicheKeys_haveCompleteCatalogAndValidRevenueStreamMappings() {
        val expectedNiches = setOf(
            "HOME_TRADES",
            "SALON_AND_CLEANERS",
            "DENTAL_AND_TUTORS",
            "AUTO_DETAILING",
            "BOUTIQUE_FITNESS",
            "ECOMMERCE_SHOPIFY",
            "SAAS_AI_REMEDIATION"
        )
        val actualNames = NicheKey.entries.map { it.name }.toSet()
        assertEquals(expectedNiches, actualNames)
```

```
        NicheKey.entries.forEach { nicheKey ->
            val preset = NicheCatalog.getPreset(nicheKey)
            assertNotNull("Preset must exist for $nicheKey", preset)
            assertTrue("Packages must not be empty for $nicheKey",
preset.packages.isNotEmpty())
            assertTrue("Add-ons must not be empty for $nicheKey",
preset.addOns.isNotEmpty())
            assertNotNull("RevenueStreamMode must be mapped for
$nicheKey", preset.revenueStreamMode)
        }
    }

    @Test
    fun pipelineStage_transitionsProgressSequentially() {
        val stages = PipelineStage.entries
        assertTrue(stages.isNotEmpty())
        for (i in 0 until stages.size - 1) {
            val current = stages[i]
            val next = current.nextStage()
            assertEquals(stages[i + 1], next)
        }
    }

    @Test
    fun
quoteAndDepositMath_respectsFirestoreBoundsAndCalculatesAccurately() {
        val setupFee = InputBoundsValidator.clampSetupFeeOffer(65_000)
// Clamped to 50,000
        val monthlyRetainer =
InputBoundsValidator.clampMonthlyRetainerOffer(7_500) // Clamped to
5,000
        val depositPercent =
InputBoundsValidator.clampDepositPercent(120) // Clamped to 100

        assertEquals(50_000, setupFee)
        assertEquals(5_000, monthlyRetainer)
        assertEquals(100, depositPercent)

        val standardDeposit = (2_000 * 50) / 100
        assertEquals(1_000, standardDeposit)
    }

    @Test
    fun
scriptGenerators_produceNonBlankScriptsAcrossAllOfferArchetypesAndGuar
drails() {
        val guardrails = ScopeGuardrailConfig(
            useExternalPaymentLinks = true,
```

```
            useExternalLeadForms = true,
            useManagedOAuth = true
        )
        NicheKey.entries.forEach { niche ->
            OfferArchetype.entries.forEach { archetype ->
                val scripts =
PitchScriptGenerator.generateAllSixScripts(
                    nicheKey = niche,
                    archetype = archetype,
                    businessName = "Apex Service Co",
                    contactName = "Jordan",
                    setupFee = 1500,
                    monthlyRetainer = 299,
                    depositPercent = 50,
                    guardrailConfig = guardrails
                )
                assertEquals("Must generate all 6 scripts", 6,
scripts.size)
                scripts.forEach { script ->
                    assertFalse("Script body should not be blank",
script.body.isBlank())
                }
            }
        }
    }

    @Test
    fun inputBoundsValidator_enforcesExactFirestoreRuleLimits() {
        assertEquals(100,
InputBoundsValidator.sanitizeBusinessName("A".repeat(150)).length)
        assertEquals(100,
InputBoundsValidator.sanitizeContactName("B".repeat(140)).length)
        assertEquals(300,
InputBoundsValidator.sanitizeWebsiteGapNote("C".repeat(400)).length)
        assertEquals(150,
InputBoundsValidator.sanitizeTagline("D".repeat(200)).length)
        assertEquals(500,
InputBoundsValidator.sanitizeNotes("E".repeat(650)).length)
    }
}
```

2. Robolectric Guest Session & ViewModel Suite (ExampleRobolectricTest.kt)

Tests LocalGuestSessionManager CRUD operations (prospect creation, stage advancement, deletion, lead capture, playbook task toggling, pending BusinessAccountProfile staging, and

Day 1 / Day 20 / Day 31 lifecycle calculation) and VibePitchViewModel in Guest Mode:

```
package com.vibepitch.studio

import android.content.Context
import androidx.test.core.app.ApplicationProvider
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertTrue
import org.junit.Before
import org.junit.Test
import org.junit.runner.RunWith
import org.robolectric.RobolectricTestRunner

@RunWith(RobolectricTestRunner::class)
class ExampleRobolectricTest {

    private lateinit var context: Context
    private lateinit var guestManager: LocalGuestSessionManager

    @Before
    fun setUp() {
        context = ApplicationProvider.getApplicationContext()
        guestManager = LocalGuestSessionManager(context)
        guestManager.clearAllGuestData()
    }

    @Test
    fun
guestSessionLifecycle_correctlyEvaluatesDay1_Day20_andDay31States() {
        val now = System.currentTimeMillis()
        val dayInMillis = 24L * 60L * 60L * 1000L

        // Day 1: Active Guest Session (Days 1-19)
        val day1Status = guestManager.evaluateLifecycleAt(
            installEpochMillis = now - (0 * dayInMillis),
            currentEpochMillis = now
        )
        assertEquals(1, day1Status.dayCount)
        assertFalse(day1Status.shouldShowSoftPrompt)
        assertFalse(day1Status.isHardGated)

        // Day 20: Dismissible Soft Upgrade Prompt (Days 20-30)
        val day20Status = guestManager.evaluateLifecycleAt(
            installEpochMillis = now - (19 * dayInMillis),
            currentEpochMillis = now
        )
        assertEquals(20, day20Status.dayCount)
```

```
        assertTrue(day20Status.shouldShowSoftPrompt)
        assertFalse(day20Status.isHardGated)

        // Day 31: Required Upgrade Gate (Day 31+)
        val day31Status = guestManager.evaluateLifecycleAt(
            installEpochMillis = now - (30 * dayInMillis),
            currentEpochMillis = now
        )
        assertEquals(31, day31Status.dayCount)
        assertTrue(day31Status.isHardGated)
    }

    @Test
    fun
guestCrudOperations_prospectsLeadsPlaybookAndPendingBusinessProfile()
{
        // 1. Create Prospect with over-length strings to verify
truncation
        val created = guestManager.addProspect(
            businessName = "X".repeat(120),
            contactName = "Y".repeat(120),
            nicheKey = NicheKey.SAAS_AI_REMEDIATION,
            websiteGapNote = "Z".repeat(350),
            setupFeeOffer = 60_000,
            monthlyRetainerOffer = 6_000,
            depositPercent = 50
        )
        assertEquals(100, created.businessName.length)
        assertEquals(100, created.contactName.length)
        assertEquals(300, created.websiteGapNote.length)
        assertEquals(50_000, created.setupFeeOffer)
        assertEquals(5_000, created.monthlyRetainerOffer)

        // 2. Advance Stage & Delete
        val advanced = guestManager.advanceProspectStage(created.id)
        assertNotNull(advanced)
        guestManager.deleteProspect(created.id)
        assertTrue(guestManager.getProspects().none { it.id ==
created.id })

        // 3. Lead Capture & Playbook Task Toggle
        val lead = guestManager.captureLead(
            businessName = "Glow Salon",
            contactName = "Maya",
            nicheKey = NicheKey.SALON_AND_CLEANERS,
            notes = "Interested in booking link integration"
        )
        assertEquals("Glow Salon", lead.businessName)
```

```
        guestManager.togglePlaybookTask("day_1_audit", completed =
true)

assertTrue(guestManager.isPlaybookTaskCompleted("day_1_audit"))

        // 4. Stage Pending BusinessAccountProfile for Cloud Migration
        guestManager.savePendingBusinessProfile(
            agencyName = "Apex Digital Studio",
            tagline = "T".repeat(200),
            primaryNiche = NicheKey.ECOMMERCE_SHOPIFY
        )
        val pendingProfile = guestManager.getPendingBusinessProfile()
        assertNotNull(pendingProfile)
        assertEquals(150, pendingProfile!!.tagline.length)
    }
}
```

3. Firestore Security Rules & Emulator Test Suite (firestore.test.js)

Verifies all 7 NicheKey values (HOME_TRADES, SALON_AND_CLEANERS, DENTAL_AND_TUTORS, AUTO_DETAILING, BOUTIQUE_FITNESS, ECOMMERCE_SHOPIFY, SAAS_AI_REMEDIATION), unauthenticated denial, cross-user isolation, and schema bounds:

```
const { test, before, after, beforeEach } = require('node:test');
const assert = require('node:assert/strict');
const fs = require('node:fs');
const {
  initializeTestEnvironment,
  assertSucceeds,
  assertFails,
} = require('@firebase/rules-unit-testing');

const PROJECT_ID = 'vibepitch-studio-test';
const ALL_NICHE_KEYS = [
  'HOME_TRADES',
  'SALON_AND_CLEANERS',
  'DENTAL_AND_TUTORS',
  'AUTO_DETAILING',
  'BOUTIQUE_FITNESS',
  'ECOMMERCE_SHOPIFY',
  'SAAS_AI_REMEDIATION',
];

let testEnv;

before(async () => {
```

```
  testEnv = await initializeTestEnvironment({
    projectId: PROJECT_ID,
    firestore: {
      rules: fs.readFileSync('firestore.rules', 'utf8'),
    },
  });
});

beforeEach(async () => {
  await testEnv.clearFirestore();
});

after(async () => {
  await testEnv.cleanup();
});

test('unauthenticated users are denied read and write access across
all collections', async () => {
  const unauthDb = testEnv.unauthenticatedContext().firestore();
  await assertFails(unauthDb.collection('users').doc('user_1').get());
  await assertFails(

unauthDb.collection('users').doc('user_1').collection('prospects').doc
('p1').set({
      businessName: 'Test Co',
    })
  );
});

test('authenticated user can write and read prospects across all 7
NicheKey values', async () => {
  const aliceDb =
testEnv.authenticatedContext('alice_uid').firestore();

  for (const nicheKey of ALL_NICHE_KEYS) {
    const docRef = aliceDb
      .collection('users')
      .doc('alice_uid')
      .collection('prospects')
      .doc(`prospect_${nicheKey}`);

    await assertSucceeds(
      docRef.set({
        ownerUid: 'alice_uid',
        businessName: `Sample ${nicheKey}`,
        contactName: 'Alex Owner',
        nicheKey: nicheKey,
        websiteGapNote: 'Missing mobile booking CTA',
```

```
        notes: 'Ready for Day 1 pitch',
        depositPercent: 50,
        setupFeeOffer: 1500,
        monthlyRetainerOffer: 250,
        stage: 'LEAD',
        updatedAt: Date.now(),
      })
    );
  }
});

test('cross-user isolation prevents bob_uid from reading or modifying
alice_uid data', async () => {
  const aliceDb =
testEnv.authenticatedContext('alice_uid').firestore();
  const bobDb = testEnv.authenticatedContext('bob_uid').firestore();

  const aliceProspectRef = aliceDb
    .collection('users')
    .doc('alice_uid')
    .collection('prospects')
    .doc('p_secret');

  await assertSucceeds(
    aliceProspectRef.set({
      ownerUid: 'alice_uid',
      businessName: 'Alice Exclusive Client',
      contactName: 'Sarah',
      nicheKey: 'HOME_TRADES',
      websiteGapNote: 'No SSL certificate',
      notes: 'Follow up Friday',
      depositPercent: 50,
      setupFeeOffer: 1200,
      monthlyRetainerOffer: 199,
      stage: 'LEAD',
      updatedAt: Date.now(),
    })
  );

  const bobAttemptRef = bobDb
    .collection('users')
    .doc('alice_uid')
    .collection('prospects')
    .doc('p_secret');

  await assertFails(bobAttemptRef.get());
  await assertFails(bobAttemptRef.delete());
});
```

Phase 4: Documentation & Global Compliance Suite (+ In-App Viewer)

1. /README.md

```
# VibePitch Studio

**VibePitch Studio** is an Android mobile sales enablement, live white-label pitch studio, quote calculator, and pipeline management platform built with Kotlin, Jetpack Compose, and Firebase Firestore. It empowers solo agencies, freelance developers, and AI remediation consultants to pitch, scope, quote, and close clients across 7 high-margin verticals without writing fragile backend payment or PII handling code.

## Core Capabilities & Multi-Stream Architecture

1. **7 Turnkey Vertical Niches (`NicheKey`)**:
   - `HOME_TRADES`: HVAC, plumbing, roofing, and electrical dispatch & instant quote funnels.
   - `SALON_AND_CLEANERS`: Recurring appointment booking and SMS reminder micro-sites.
   - `DENTAL_AND_TUTORS`: High-trust patient/student intake and scheduling portals.
   - `AUTO_DETAILING`: Tiered ceramic coating & mobile detailing package configurators.
   - `BOUTIQUE_FITNESS`: Class pass, trial membership, and trainer booking funnels.
   - `ECOMMERCE_SHOPIFY`: Conversion rate optimization, speed remediation, and bundle upsells.
   - `SAAS_AI_REMEDIATION`: Vibe-coded app auditing, security rules hardening, and production refactoring.

2. **Interactive Quote & Scope Guardrail Engine**:
   - Real-time calculation of Setup Fees (`0–50,000`), Monthly Retainers (`0–5,000`), and Deposit Splits (`0%–100%`).
   - Built-in **Scope Guardrails** enforcing third-party offloading for payments (Stripe Payment Links), PII intake (Google Forms / Typeform), and authentication (Managed OAuth / Firebase Auth).

3. **6-Part Pitch & Proposal Script Generator**:
   - Automatically generates tailored Cold Openers, Website Gap Audits, Live Demo Walk-Throughs, Objection Handlers, Scope-Safe Proposals, and Referral Ask scripts across multiple `OfferArchetype`
```

models.

4. **30-Day Guest Account to Permanent Business Account Lifecycle**:
   - **Days 1–19**: Frictionless on-device Guest Mode via `LocalGuestSessionManager` with zero sign-up barrier.
   - **Days 20–30**: Dismissible upgrade banner prompting users to bind a permanent Google Business Account.
   - **Day 31+**: Required upgrade gate that automatically migrates all local prospects, leads, playbook progress, and branding to Firebase Cloud Firestore (`migrateGuestDataToCloud`) upon Google Sign-In via Android Jetpack `CredentialManager`.

Tech Stack

- **Language & UI**: Kotlin 2.x, Jetpack Compose (Material 3), Coroutines & StateFlow
- **Android SDK Matrix**: `compileSdk 36`, `targetSdk 36`, `minSdk 24`
- **Authentication**: AndroidX `CredentialManager` + Google ID Token Sign-In (`GoogleAuthHelper`)
- **Cloud Database**: Firebase Cloud Firestore with zero-trust owner-isolated `firestore.rules`
- **Local Persistence**: `SharedPreferences` JSON state engine (`LocalGuestSessionManager`)
- **Testing**: JUnit 4, Robolectric, Firebase Local Emulator Suite (`@firebase/rules-unit-testing`)

Prerequisites & Configuration

1. **Android Studio**: Ladybug / Meerkat or newer with JDK 17 configured.
2. **Node.js**: v20+ and Firebase CLI (`npm install -g firebase-tools`) for running Firestore security rule tests.
3. **Firebase Project Configuration**:
   - Place your `google-services.json` inside `app/google-services.json`.
   - Enable **Google Sign-In** in the Firebase Authentication console and configure your Web Client ID in `strings.xml` or runtime environment configuration.

Build & Test Commands

```bash
# 1. Run Tier 1 Firestore Security Rules Emulator Tests
node --test firestore.test.js

# 2. Run Tier 2 Kotlin Unit & Robolectric Tests
./gradlew :app:testDebugUnitTest
```

```
# 3. Assemble Debug APK
./gradlew :app:assembleDebug
```

Compliance & Governance Documentation

All repository governance, cybersecurity, and privacy documents are available in the root directory and inside the app via the **Docs & Compliance** button:

- SECURITY.md — Vulnerability reporting, encryption, and zero-trust Firestore architecture.
- PRIVACY.md — Global privacy compliance (150+ jurisdictions as of October 10, 2026).
- TERMS_AND_CONDITIONS.md — Commercial usage terms, white-label demo disclaimers, and liability limits.
- ACCOUNT.md — 30-day Guest lifecycle, Google OAuth binding, RBAC, and account deletion flows.
- CONTRIBUTING.md — Branching workflow, commit standards, and PR verification requirements.
- CODE_OF_CONDUCT.md — Contributor Covenant v2.1 community guidelines.
- ROADMAP.md — Q4 2026, 2027, and Long-Term technical milestones.

```
### 2. `/SECURITY.md`

```markdown
# Security Policy & Architecture (`SECURITY.md`)

**Effective Date:** October 10, 2026
**Status:** Active Enforcement

## 1. Supported Versions

Security patches, dependency updates, and Firestore rule updates are
actively maintained for the following release branches:

| Version | Android API Level | Support Status |
| :--- | :--- | :--- |
| `1.1.x` (Current) | API 24 - API 36 | Actively Supported |
| `1.0.x` | API 24 - API 35 | Critical Security Fixes Only |
| `< 1.0.0` | Legacy | Unsupported |

## 2. Zero-Trust Firestore Security Architecture

VibePitch Studio enforces strict document-level isolation and schema
validation in `firestore.rules`:

1. **Default-Deny Posture**: All paths not explicitly matched in
`firestore.rules` are rejected unconditionally.
2. **Strict Owner Isolation**: Every user document and subcollection
(`prospects`, `leads`, `playbook`, `profile`) resides under
`/users/{userId}` and requires `request.auth != null &&
```
```

request.auth.uid == userId`.
3. **Server-Side Schema & Bounds Validation**:
   - `nicheKey` is strictly validated against the 7 allowed vertical constants (`HOME_TRADES`, `SALON_AND_CLEANERS`, `DENTAL_AND_TUTORS`, `AUTO_DETAILING`, `BOUTIQUE_FITNESS`, `ECOMMERCE_SHOPIFY`, `SAAS_AI_REMEDIATION`).
   - String lengths are capped at the rule layer (`businessName <= 100`, `contactName <= 100`, `websiteGapNote <= 300`, `tagline <= 150`, `notes <= 500`) to prevent payload bloat and storage exhaustion attacks.
   - Financial fields are bounded (`depositPercent` in `0..100`, `setupFeeOffer` in `0..50000`, `monthlyRetainerOffer` in `0..5000`).

3. Payment & PII Offloading Guardrails

By architectural design, VibePitch Studio **never** processes, transmits, or stores raw payment card numbers (PAN), CVV codes, bank routing numbers, or sensitive end-customer health/financial records:
- **Payments**: Offloaded exclusively to PCI-DSS Level 1 compliant external links (e.g., Stripe Payment Links).
- **Customer Intake**: Offloaded to dedicated external form providers (e.g., Google Forms, Typeform, Calendly).
- **Authentication**: Handled via AndroidX `CredentialManager` and Firebase Authentication using cryptographic OAuth 2.0 / OpenID Connect ID tokens.

4. Encryption Standards

- **Data in Transit**: All communication with Firebase Authentication and Cloud Firestore is encrypted via TLS 1.3.
- **Data at Rest (Cloud)**: All Firestore documents are encrypted at rest using AES-256 managed by Google Cloud Platform.
- **Data at Rest (On-Device Guest Mode)**: Guest session data resides within the private Android application sandbox (`/data/data/com.vibepitch.studio/shared_prefs/`), protected by Android OS file-based encryption (FBE) and UID sandboxing.

5. Vulnerability Disclosure & Incident Response SLAs

If you discover a security vulnerability in VibePitch Studio or its `firestore.rules`, do **not** open a public GitHub issue. Report it privately via email to `security@vibepitch.studio` with:
- A clear description of the vulnerability and affected component.
- Steps to reproduce or a proof-of-concept test case (`firestore.test.js` or Robolectric test).

Response Timeline SLAs
- **Initial Acknowledgment**: Within **24 hours** of receipt.

- **Triage & Severity Assessment (CVSS v4.0)**: Within **72 hours**.
- **Critical/High Patch Deployment**: Within **7 days**.
- **Medium/Low Patch Deployment**: Within **30 days**.

3. /PRIVACY.md

```
# Global Privacy Policy (`PRIVACY.md`)

**Effective Date:** October 10, 2026
**Compliance Scope:** 150+ International Jurisdictions & ISO/IEC 27701:2025 Privacy Information Management System (PIMS)

## 1. Regulatory Alignment & Global Scope

This Privacy Policy governs the collection, processing, storage, and transfer of personal data by **VibePitch Studio** in accordance with international cybersecurity and data protection laws in effect as of **October 10, 2026**, including but not limited to:
- **European Economic Area (EEA) & UK**: General Data Protection Regulation (EU GDPR) and UK Data Protection Act / UK GDPR.
- **United States**: California Consumer Privacy Act as amended by the California Privacy Rights Act (CCPA/CPRA), Virginia CDPA, Colorado CPA, Connecticut CTDPA, Texas TDPSA, and applicable state privacy statutes.
- **India**: Digital Personal Data Protection Act, 2023 (DPDP Act) and enforcing rules.
- **Brazil**: Lei Geral de Proteção de Dados Pessoais (LGPD — Law No. 13.709/2018).
- **People's Republic of China**: Personal Information Protection Law (PIPL), Cybersecurity Law (CSL), and Data Security Law (DSL).
- **Canada**: Personal Information Protection and Electronic Documents Act (PIPEDA) and Quebec Law 25.
- **Asia-Pacific, Middle East & Africa**: Australia Privacy Act 1988 (as amended), Japan APPI, Singapore PDPA, South Korea PIPA, UAE Federal Decree-Law No. 45 of 2021, Saudi Arabia PDPL, South Africa POPIA, Nigeria NDPA, and over 130 additional national data protection frameworks aligned with the Berne/OECD/Council of Europe Convention 108+ principles.

## 2. Categories of Data Processed

### A. On-Device Guest Account Mode (Days 1-30)
When operating in Guest Mode prior to Google Sign-In:
- **Data Collected**: Local prospect business names, prospect contact names, website gap notes, quote configurations, playbook checklist toggles, and installation timestamp (`installEpochMillis`).
- **Storage Location**: Stored strictly locally on your Android device
```

via `LocalGuestSessionManager`. **Zero personal data is transmitted to cloud servers during Guest Mode.**

B. Permanent Business Account Mode (Authenticated via Google Sign-In)
When you link your Google Account via Android `CredentialManager`:
- **Authentication Identifiers**: Firebase User UID (`uid`), email address, and display name provided via Google OAuth consent.
- **Cloud-Synced Business Records**: Agency branding profile (`BusinessAccountProfile`), B2B prospect pipeline entries, captured lead notes, and playbook progress stored under `/users/{uid}` in Firebase Cloud Firestore.

C. Prohibited Sensitive Personal Data
VibePitch Studio does not request, collect, or process Sensitive Personal Data / Special Category Data (such as biometric identifiers, health records, government ID numbers, payment card credentials, or precise background geolocation). Users are prohibited from entering sensitive personal data into free-text notes fields.

3. Lawful Basis for Processing

Depending on your jurisdiction, we process personal data under the following legal bases:
1. **Performance of a Contract**: Providing local and cloud-synced sales pipeline, quoting, and pitch script functionality requested by the user.
2. **Explicit Consent**: Obtained when you initiate Google Sign-In or trigger Guest-to-Cloud migration (`migrateGuestDataToCloud`).
3. **Legitimate Interests**: Maintaining application security, enforcing `firestore.rules` tenant isolation, and preventing abuse.

4. Data Retention & Deletion

- **Guest Mode Data**: Retained solely on-device until cleared by the user, uninstalled, or migrated to a Permanent Business Account.
- **Permanent Business Account Data**: Retained in Firebase Cloud Firestore only while your account remains active.
- **Immediate Erasure**: Users may delete individual prospects/leads at any time or execute a full account and cloud data deletion (`ACCOUNT.md`), which permanently purges `/users/{uid}` and all nested subcollections within **72 hours** (and from encrypted backups within **30 days**).

5. International Data Transfers

Cloud data synced via Firebase Cloud Firestore is processed in Google Cloud data centers certified under **ISO/IEC 27001**, **ISO/IEC

```
27018**, and **ISO/IEC 27701**. Cross-border transfers rely on:
- **EU-U.S. Data Privacy Framework (DPF)**, UK Extension, and
Swiss-U.S. DPF.
- **European Commission Standard Contractual Clauses (SCCs)** and UK
International Data Transfer Addendum (IDTA).
- Standard contractual transfer mechanisms recognized under India DPDP
Act, Brazil LGPD, and ASEAN Model Contractual Clauses.

## 6. Your Global Privacy Rights

Regardless of your country of residence, VibePitch Studio extends the
highest tier of statutory privacy rights to all users:
- **Right of Access & Portability**: Export your complete profile,
prospects, and leads in structured JSON format.
- **Right to Rectification**: Edit any prospect, lead, or agency
profile field directly in the app.
- **Right to Erasure ("Right to be Forgotten")**: Clear local guest
storage or permanently delete your cloud account and all associated
records.
- **Right to Withdraw Consent & Restrict Processing**: Sign out or
revoke Google OAuth permissions at any time via Google Account
Security settings.
- **Right to Non-Discrimination & Automated Decision Opt-Out**:
VibePitch Studio does not sell personal data, share data for
cross-context behavioral advertising, or perform automated profiling
with legal effects.

## 7. Data Protection Officer (DPO) & Grievance Officer Contact

For privacy inquiries, Data Subject Access Requests (DSARs), or India
DPDP Act Grievance Redressal:
- **Email**: `privacy@vibepitch.studio`
- **Response SLA**: Acknowledgment within **48 hours**; full
resolution within **15 calendar days** (or shorter where mandated by
local statute).
```

4. /TERMS_AND_CONDITIONS.md

```
# Terms and Conditions of Service (`TERMS_AND_CONDITIONS.md`)

**Effective Date:** October 10, 2026

## 1. Acceptance of Terms

By downloading, installing, accessing, or using **VibePitch Studio**
(the "Application"), whether in 30-Day Guest Mode or via an
authenticated Permanent Business Account, you agree to be bound by
```

these Terms and Conditions ("Terms"). If you do not agree to these Terms, do not use the Application.

2. License Grant & White-Label Pitch Usage

Subject to your compliance with these Terms, you are granted a revocable, non-exclusive, non-transferable license to use the Application to:
1. Present interactive white-label product previews and pricing configurations to prospective business clients across the 7 supported verticals.
2. Generate, customize, and send pitch scripts, scope guardrails, and proposals created within the Application for your legitimate commercial agency or freelance consulting operations.

3. User Obligations & Scope Guardrail Compliance

You expressly agree that:
1. **Lawful Outreach**: Any cold outreach, SMS, email, or call scripts generated by the Application must be used in compliance with applicable telemarketing and anti-spam laws in your jurisdiction (including CAN-SPAM, TCPA, GDPR ePrivacy Directive, CASL, and TRAI regulations).
2. **External Payment & PII Offloading**: You are solely responsible for configuring your own external payment processors (e.g., Stripe Payment Links) and intake forms when delivering projects to your clients. VibePitch Studio is not a payment processor, escrow agent, or merchant of record.
3. **Prohibited Content**: You will not store unlawful, defamatory, infringing, or regulated sensitive personal data (such as medical records or payment card numbers) inside the Application's prospect or lead notes.

4. 30-Day Guest Trial & Permanent Business Account Conversion

- The Application provides a **30-day local Guest Account lifecycle** (`Days 1-19` active use, `Days 20-30` dismissible upgrade reminders).
- Beginning on **Day 31**, continued access to pipeline creation and editing requires linking a Google Account via Android `CredentialManager` to transition to a Permanent Business Account.
- Prior to linking a Permanent Business Account, all Guest Mode data resides solely on your local device; clearing app storage or uninstalling the Application prior to cloud migration will permanently erase unmigrated Guest data.

5. Intellectual Property Rights

- **Application Code & Architecture**: The VibePitch Studio source

code, UI layout, domain catalogs (`NicheCatalog`), and Firestore rules remain the exclusive property of the repository maintainers and contributors as licensed under the repository license.
- **Your Business Data**: You retain 100% ownership of the custom agency branding, prospect lists, pricing configurations, and client notes you create in the Application.

6. Disclaimer of Warranties & Limitation of Liability

THE APPLICATION IS PROVIDED "AS IS" AND "AS AVAILABLE" WITHOUT WARRANTIES OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. WE DO NOT GUARANTEE THAT USE OF THE PITCH SCRIPTS OR DEMO PREVIEWS WILL RESULT IN SPECIFIC CLIENT CONVERSIONS OR REVENUE LEVELS.

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, IN NO EVENT SHALL THE AUTHORS, MAINTAINERS, OR CONTRIBUTORS BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, OR PUNITIVE DAMAGES, OR ANY LOSS OF PROFITS, REVENUE, OR DATA ARISING FROM YOUR USE OF THE APPLICATION.

7. Governing Law & Dispute Resolution

These Terms shall be governed by the applicable laws of the jurisdiction in which the primary entity or developer operates, without regard to conflict-of-law principles, while preserving any mandatory consumer protection rights guaranteed by your country of residence. Any dispute shall first be submitted to good-faith informal resolution via `legal@vibepitch.studio` for thirty (30) days prior to formal arbitration or litigation.

5. /ACCOUNT.md

Account Lifecycle, Authentication & Access Control (`ACCOUNT.md`)

Last Updated: October 10, 2026

1. Dual-Mode Account Architecture

VibePitch Studio implements a progressive onboarding model designed to deliver immediate utility on Day 1 while ensuring durable cloud backup and multi-device continuity for long-term commercial users:

1. **Local Guest Account (`LocalGuestSessionManager`)**: Instant on-device workspace requiring zero registration.
2. **Permanent Business Account (`GoogleAuthHelper` + `VibePitchRepository`)**: Cloud-backed workspace authenticated via

Google OAuth and isolated in Firebase Cloud Firestore.

2. The 30-Day Guest Account Lifecycle

The guest lifecycle is calculated deterministically from the initial installation timestamp (`installEpochMillis`):

Lifecycle Window	Day Count	UI & Functional Behavior
Active Guest Window	Days 1 - 19	Full local access to all 7 vertical demos, quote calculator, pitch scripts, prospect pipeline, and playbook checklist.
Soft Upgrade Window	Days 20 - 30	Displays a dismissible upgrade banner reminding the user to link their Google Account and stage their `BusinessAccountProfile`.
Required Upgrade Gate	Day 31+	Displays a required upgrade modal gating further local mutations until the user completes Google Sign-In, automatically migrating all guest data to the cloud.

3. Authentication Flow (Jetpack `CredentialManager`)

1. The user taps **Upgrade to Business Account** or **Sign in with Google**.
2. `GoogleAuthHelper` invokes the AndroidX `CredentialManager` API with `GetGoogleIdOption`.
3. Upon user consent, Google returns a signed OIDC ID Token.
4. `VibePitchRepository` exchanges the Google ID Token with `FirebaseAuth.getInstance().signInWithCredential(GoogleAuthProvider.getCredential(idToken, null))`.
5. Once authenticated, `FirebaseAuth.currentUser.uid` becomes the immutable owner identifier (`ownerUid`).

4. Automatic Guest-to-Cloud Data Migration (`migrateGuestDataToCloud`)

Immediately after a Guest user authenticates with Google for the first time, `VibePitchRepository.migrateGuestDataToCloud()` executes an atomic batch migration:
1. Reads all local prospects, captured leads, playbook task states, and any staged `BusinessAccountProfile` from `LocalGuestSessionManager`.
2. Sanitizes every string and numeric field through `InputBoundsValidator` to guarantee compliance with `firestore.rules`.
3. Writes all records into `/users/{uid}/profile`, `/users/{uid}/prospects/{prospectId}`, `/users/{uid}/leads/{leadId}`, and `/users/{uid}/playbook/{taskId}`.
4. Marks the local guest migration flag as completed and clears redundant staging buffers.

```
## 5. Role-Based Access Control (RBAC) & Tenant Isolation

VibePitch Studio enforces strict single-tenant owner isolation via
Firebase Security Rules (`firestore.rules`):
- **Owner Role (`request.auth.uid == userId`)**: Full read, create,
update, and delete privileges on documents inside
`/users/{userId}/**`, subject to schema validation rules.
- **Cross-Tenant Users (`request.auth.uid != userId`)**: Zero read or
write access. Every cross-user query or mutation is rejected at the
Firestore edge.
- **Unauthenticated Requests (`request.auth == null`)**: Rejected
across all cloud collections.

## 6. Account Deletion & Data Export Procedures

### Exporting Your Account Data
- Users can copy individual or complete pipeline summaries and pitch
scripts directly to the Android clipboard or share sheet at any time.

### Deleting Local Guest Data
- In Guest Mode, open **Playbook / Settings -> Clear Local Guest
Workspace** (or clear application storage in Android Settings) to
immediately wipe all on-device `SharedPreferences` records.

### Deleting Your Permanent Business Account
1. While signed in, select **Account -> Delete Permanent Business
Account** or email `privacy@vibepitch.studio` from your registered
Google email address.
2. The application deletes all documents under `/users/{uid}`
(`prospects`, `leads`, `playbook`, `profile`), revokes the Firebase
Authentication user record, and resets the local device workspace.
```

6. /CONTRIBUTING.md

```
# Contributing to VibePitch Studio (`CONTRIBUTING.md`)

We welcome contributions that improve VibePitch Studio's vertical demo
presets, pitch script generators, Jetpack Compose UI, and Firestore
security rules.

## 1. Branching Strategy

- `main`: Protected production branch. All merges require passing Tier
1 (Firestore Emulator) and Tier 2 (Gradle Unit + Robolectric) test
suites.
- `feat/<short-slug>`: New vertical features, script enhancements, or
```

UI screens.
- `fix/<short-slug>`: Bug fixes, bounds hardening, or coroutine cancellation fixes.
- `docs/<short-slug>`: Documentation, compliance, and roadmap updates.

2. Conventional Commit Format

Follow the [Conventional Commits](https://www.conventionalcommits.org/) specification:
- `feat(catalog): add seasonal upsell add-on to AUTO_DETAILING preset`
- `fix(viewmodel): clamp setupFeeOffer to 50000 before Firestore write`
- `test(rules): verify all 7 NicheKey values in firestore.test.js`
- `docs(privacy): align PRIVACY.md with October 2026 global frameworks`

3. Coding Standards & Architectural Guardrails

1. **Kotlin & Jetpack Compose**:
   - Target JDK 17 and maintain compatibility with `minSdk 24` through `compileSdk 36`.
   - Lift UI state to `VibePitchViewModel` via immutable `StateFlow` snapshots; keep Compose screen composables stateless wherever possible.
   - Never swallow `CancellationException` inside `try/catch` blocks within coroutines.
2. **Firestore Schema Synchronization**:
   - Any change to model fields in `VibePitchModels.kt` **must** be accompanied by matching bounds validation in `InputBoundsValidator.kt`, `firestore.rules`, and `firestore.test.js`.
3. **Zero Raw Payment/PII Handling**:
   - Do not introduce SDKs or UI fields that collect raw credit card numbers, SSNs/Aadhaar numbers, or passwords. Always use the Scope Guardrail pattern (external Stripe Payment Links, external intake forms, and managed Google OAuth).

4. Local Setup & Pre-PR Verification Checklist

Before opening a Pull Request, run the complete verification suite locally:

```bash
# Step 1: Verify Firestore Security Rules against the emulator
node --test firestore.test.js

# Step 2: Run Unit & Robolectric Tests
./gradlew :app:testDebugUnitTest
```

```
# Step 3: Verify Debug APK Compilation
./gradlew :app:assembleDebug
```

All Pull Requests must include:

- A concise summary of architectural changes.
- Updated unit or Robolectric tests in ExampleUnitTest.kt / ExampleRobolectricTest.kt covering new functions and edge cases.
- Screenshots or screen recordings for any Jetpack Compose UI modifications.

```
### 7. `/CODE_OF_CONDUCT.md`

```markdown
# Contributor Covenant Code of Conduct (`CODE_OF_CONDUCT.md`)

**Version:** 2.1 (Adapted for VibePitch Studio)

## 1. Our Pledge

We as members, contributors, and maintainers of **VibePitch Studio**
pledge to make participation in our community a harassment-free
experience for everyone, regardless of age, body size, visible or
invisible disability, ethnicity, sex characteristics, gender identity
and expression, level of experience, education, socio-economic status,
nationality, personal appearance, race, caste, color, religion, or
sexual identity and orientation.

We pledge to act and interact in ways that contribute to an open,
welcoming, diverse, inclusive, and healthy engineering community.

## 2. Our Standards

Examples of behavior that contributes to a positive environment
include:
- Demonstrating empathy, candor, and constructive technical feedback
during code reviews.
- Respecting differing perspectives, business verticals, and regional
markets.
- Giving and gracefully accepting constructive feedback on
architecture, security rules, and test coverage.
- Focusing on what is best for the security, privacy, and reliability
of the community and end users.

Examples of unacceptable behavior include:
- The use of sexualized language or imagery, and sexual attention or
advances of any kind.
- Trolling, insulting or derogatory comments, and personal or
political attacks.
- Public or private harassment.
- Publishing others' private information, such as a physical or
```
```

```
electronic address, or committing real API keys/secrets without
explicit permission.
- Intentionally introducing backdoors, malicious dependencies, or
weakened security rules (`allow read, write: if true;`).

## 3. Enforcement Responsibilities

Repository maintainers are responsible for clarifying and enforcing
our standards of acceptable behavior and will take appropriate and
fair corrective action in response to any behavior that they deem
inappropriate, threatening, offensive, or harmful.

## 4. Scope

This Code of Conduct applies within all project spaces (source code,
issues, pull requests, discussions, and documentation) and also
applies when an individual is officially representing the project in
public spaces.

## 5. Reporting & Enforcement Guidelines

Instances of abusive, harassing, or otherwise unacceptable behavior
may be reported to the project maintainers at
`conduct@vibepitch.studio`. All complaints will be reviewed and
investigated promptly and fairly:
1. **Correction**: A private, written warning providing clarity around
the nature of the violation.
2. **Warning**: A formal warning with consequences for continued
behavior, including temporary restriction from pull request or issue
interaction.
3. **Permanent Ban**: Permanent removal from repository participation
for severe or repeated violations.
```

8. /ROADMAP.md

```
# Product & Technical Roadmap (`ROADMAP.md`)

**Last Updated:** Q4 2026

## 1. Short-Term Milestones (Q4 2026 — Completed & Active)

- [x] **7-Vertical Niche Catalog (`NicheCatalog`)**: Complete
interactive demo presets, packages, and add-ons for `HOME_TRADES`,
`SALON_AND_CLEANERS`, `DENTAL_AND_TUTORS`, `AUTO_DETAILING`,
`BOUTIQUE_FITNESS`, `ECOMMERCE_SHOPIFY`, and `SAAS_AI_REMEDIATION`.
- [x] **6-Part Pitch & Scope Script Generator**: Dynamic generation of
cold openers, website gap audits, demo scripts, objection handlers,
```

scope-safe proposals, and referral scripts.
- [x] **30-Day Guest-to-Business Account Lifecycle**: Local `LocalGuestSessionManager` with Day 20–30 soft upgrade prompts, Day 31+ hard gate, Jetpack `CredentialManager` Google Sign-In, and automatic cloud migration (`migrateGuestDataToCloud`).
- [x] **Zero-Trust Firestore Rules & Input Hardening**: Strict owner isolation (`request.auth.uid == userId`), enum validation for all 7 `NicheKey` values, and client/server bounds alignment (`InputBoundsValidator`).
- [x] **Global Compliance & In-App Docs Hub**: Full suite of 8 repository governance/compliance documents and interactive `ComplianceDocsDialog.kt` viewer inside the app.

2. Mid-Term Milestones (2027)

- [ ] **PDF & Branded HTML Proposal Export (Q1 2027)**: Generate one-tap shareable PDF proposals containing the prospect's custom quote breakdown, selected add-ons, Scope Guardrail disclaimers, and embedded Stripe Payment Link QR codes.
- [ ] **Multi-Currency & Regional Tax Presets (Q1 2027)**: Support instant currency formatting and tax/GST/VAT toggles across USD (`$`), EUR (`€`), GBP (`£`), INR (`₹`), BRL (`R$`), CAD (`CA$`), and AUD (`A$`).
- [ ] **Offline-First WorkManager Cloud Sync Queue (Q2 2027)**: Enhance Permanent Business Accounts with background conflict resolution and retry telemetry when pitching in low-connectivity field environments.
- [ ] **Custom Vertical Preset Builder (Q3 2027)**: Allow authenticated Business Account users to clone any of the 7 core verticals and save custom package tiers and industry terminology in `/users/{uid}/customPresets`.
- [ ] **Playbook Analytics & Conversion Funnel Insights (Q4 2027)**: On-device and owner-isolated pipeline velocity metrics (Lead -> Pitched -> Proposal Sent -> Deposit Paid conversion rates by vertical).

3. Long-Term Vision (2028 & Beyond)

- [ ] **Multi-Seat Agency Team Workspaces**: Role-based team collaboration (`OWNER`, `CLOSER`, `FULFILLMENT`) with dedicated Firestore security rules for shared agency pipelines.
- [ ] **Client-Facing Interactive Micro-Preview Links**: Publish read-only, time-expiring web preview snapshots of a configured vertical pitch so prospects can review their custom package asynchronously on desktop or mobile.
- [ ] **Automated Website Gap Scanner Integration**: Optional lightweight URL metadata inspection (SSL status, mobile viewport check, booking CTA detection) to pre-populate the `websiteGapNote` field automatically.

9. In-App Compliance, Security & Legal Docs Hub (ComplianceDocsDialog.kt)

Adds the interactive **Docs & Compliance** viewer accessible from the top bar and Playbook screen so users can browse all 8 documents (README, SECURITY, PRIVACY, TERMS, ACCOUNT, CONTRIBUTING, CODE OF CONDUCT, and ROADMAP) directly inside the Android app:

```
package com.vibepitch.studio.ui.components

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.ExperimentalLayoutApi
import androidx.compose.foundation.layout.FlowRow
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.heightIn
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.FilterChip
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp

enum class ComplianceDocTab(
    val label: String,
    val fileName: String,
    val summaryTitle: String,
    val content: String
) {
    README(
        label = "README",
        fileName = "README.md",
```

```
        summaryTitle = "VibePitch Studio Architecture & Overview",
        content = """
            • Multi-Stream Sales & Pitch Studio supporting 7 turnkey
verticals (HOME_TRADES, SALON_AND_CLEANERS, DENTAL_AND_TUTORS,
AUTO_DETAILING, BOUTIQUE_FITNESS, ECOMMERCE_SHOPIFY,
SAAS_AI_REMEDIATION).
            • Real-time Quote & Deposit Calculator (0–50,000 setup fee,
0–5,000/mo retainer, 0%–100% deposit).
            • 6-Part Pitch, Proposal, Scope Guardrail & Referral
Script Generator.
            • 30-Day Local Guest Account to Permanent Cloud Business
Account lifecycle.
        """.trimIndent()
    ),
    SECURITY(
        label = "SECURITY",
        fileName = "SECURITY.md",
        summaryTitle = "Zero-Trust Firestore & Payment/PII
Offloading",
        content = """
            • Zero-Trust Firestore Rules: Strict owner isolation
(/users/{userId}) and server-side bounds validation.
            • Payment & PII Offloading Guardrails: Never stores raw
credit cards, banking credentials, or sensitive PII. Payments use
external Stripe Links; intake uses external forms; auth uses Google
OAuth via Jetpack CredentialManager.
            • Encryption: TLS 1.3 in transit, AES-256 at rest in Cloud
Firestore, and Android OS sandbox encryption in Guest Mode.
            • Vulnerability SLA: 24h acknowledgment, 72h triage, 7d
critical patch deployment.
        """.trimIndent()
    ),
    PRIVACY(
        label = "PRIVACY",
        fileName = "PRIVACY.md",
        summaryTitle = "150+ Country Privacy Compliance (Oct 10,
2026)",
        content = """
            • Aligned with GDPR, UK GDPR, CCPA/CPRA, India DPDP Act,
Brazil LGPD, China PIPL, Canada PIPEDA, Australia Privacy Act, and
ISO/IEC 27701.
            • Guest Mode (Days 1–30): 100% on-device storage via
LocalGuestSessionManager; zero cloud transmission.
            • Permanent Business Account: Syncs only your agency
profile, B2B prospects, leads, and playbook tasks under /users/{uid}.
            • Global User Rights: Full access, rectification, JSON
export, and permanent deletion within 72 hours.
        """.trimIndent()
```

```
    ),
    TERMS(
        label = "TERMS",
        fileName = "TERMS_AND_CONDITIONS.md",
        summaryTitle = "Commercial Usage & Scope Guardrail Terms",
        content = """
            • Grants a commercial license to present white-label
demos, calculate quotes, and send generated proposals to prospective
clients.
            • Users are responsible for complying with local outreach
laws and configuring their own external Stripe Payment Links and
intake forms.
            • Prohibits storing sensitive personal data or unlawful
content in free-text prospect notes.
        """.trimIndent()
    ),
    ACCOUNT(
        label = "ACCOUNT",
        fileName = "ACCOUNT.md",
        summaryTitle = "30-Day Guest Lifecycle & Google Cloud
Migration",
        content = """
            • Days 1–19: Full frictionless local Guest Mode.
            • Days 20–30: Dismissible banner to stage
BusinessAccountProfile and link Google Account.
            • Day 31+: Required upgrade gate that automatically
migrates all local guest data to Cloud Firestore
(migrateGuestDataToCloud) via Jetpack CredentialManager Google
Sign-In.
            • Instant self-service local wipe or permanent cloud
account deletion.
        """.trimIndent()
    ),
    CONTRIBUTING(
        label = "CONTRIBUTING",
        fileName = "CONTRIBUTING.md",
        summaryTitle = "Engineering Standards & Emulator
Verification",
        content = """
            • Conventional Commits and feature-branch workflow.
            • Schema synchronization required across
VibePitchModels.kt, InputBoundsValidator.kt, firestore.rules, and
firestore.test.js.
            • All PRs must pass Tier 1 (node --test firestore.test.js)
and Tier 2 (./gradlew :app:testDebugUnitTest).
        """.trimIndent()
    ),
    CODE_OF_CONDUCT(
```

```
        label = "CODE OF CONDUCT",
        fileName = "CODE_OF_CONDUCT.md",
        summaryTitle = "Contributor Covenant v2.1 Standards",
        content = """
            • Pledges an inclusive, harassment-free community for all
contributors and users.
            • Prohibits harassment, doxxing, or intentionally
weakening security rules or committing secrets.
            • Clear 3-tier enforcement model (Correction, Warning,
Permanent Ban) via conduct@vibepitch.studio.
        """.trimIndent()
    ),
    ROADMAP(
        label = "ROADMAP",
        fileName = "ROADMAP.md",
        summaryTitle = "Q4 2026, 2027 & Long-Term Product Roadmap",
        content = """
            • Short-Term (Q4 2026): 7-vertical catalog, 6-script
generator, 30-day guest-to-cloud lifecycle, zero-trust Firestore
rules, and 8-doc compliance suite.
            • Mid-Term (2027): Branded PDF proposal export with QR
payment links, multi-currency presets (USD/EUR/GBP/INR/BRL), offline
sync queue, and custom vertical builder.
            • Long-Term (2028+): Multi-seat agency workspaces and
shareable client micro-preview links.
        """.trimIndent()
    )
}

@OptIn(ExperimentalLayoutApi::class)
@Composable
fun ComplianceDocsDialog(
    onDismiss: () -> Unit
) {
    var selectedTab by rememberSaveable {
mutableStateOf(ComplianceDocTab.README) }

    AlertDialog(
        onDismissRequest = onDismiss,
        title = {
            Column {
                Text(
                    text = "Docs & Global Compliance Hub",
                    style = MaterialTheme.typography.titleLarge,
                    fontWeight = FontWeight.Bold
                )
                Text(
                    text = "ISO/IEC 27701 & 150+ Country
```

```
Cybersecurity/Privacy Suite (Oct 10, 2026)",
                    style = MaterialTheme.typography.labelMedium,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }
        },
        text = {
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .heightIn(max = 460.dp)
            ) {
                FlowRow(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement =
Arrangement.spacedBy(6.dp),
                    verticalArrangement = Arrangement.spacedBy(4.dp)
                ) {
                    ComplianceDocTab.entries.forEach { tab ->
                        FilterChip(
                            selected = selectedTab == tab,
                            onClick = { selectedTab = tab },
                            label = { Text(tab.label, style =
MaterialTheme.typography.labelSmall) }
                        )
                    }
                }

                Spacer(modifier = Modifier.height(12.dp))

                Surface(
                    tonalElevation = 2.dp,
                    shape = MaterialTheme.shapes.medium,
                    modifier = Modifier
                        .fillMaxWidth()
                        .weight(1f, fill = false)
                ) {
                    Column(
                        modifier = Modifier
                            .padding(14.dp)
                            .verticalScroll(rememberScrollState())
                    ) {
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement =
Arrangement.SpaceBetween
                        ) {
                            Text(
```

```
                                text = selectedTab.summaryTitle,
                                style =
MaterialTheme.typography.titleMedium,
                                fontWeight = FontWeight.SemiBold
                            )
                        }
                        Text(
                            text = "Root File:
/${selectedTab.fileName}",
                            style =
MaterialTheme.typography.labelSmall,
                            color = MaterialTheme.colorScheme.primary
                        )
                        Spacer(modifier = Modifier.height(8.dp))
                        Text(
                            text = selectedTab.content,
                            style =
MaterialTheme.typography.bodyMedium
                        )
                    }
                }
            }
        },
        confirmButton = {
            TextButton(onClick = onDismiss) {
                Text("Close")
            }
        }
    )
}
```